

推荐系统大作业代码与可执行文件使用说明

1. 项目文件结构

本项目用于完成“大数据计算及应用”课程推荐系统任务：根据 `train.txt` 中已有的用户评分数据，预测 `test.txt` 中用户-物品对的评分，并生成符合格式要求的 `Result.txt`。

推荐的目录结构如下：

```
任务一/  
├─ bigdata_recommender.py  
├─ recommender.exe  
├─ README.txt  
└─ data/  
    ├─ train.txt  
    ├─ test.txt  
    ├─ ResultForm.txt  
    └─ DataFormatExplanation.txt
```

其中：

<code>bigdata_recommender.py</code>	源代码文件
<code>recommender.exe</code>	已打包好的可执行文件
<code>data/train.txt</code>	训练数据
<code>data/test.txt</code>	测试数据
<code>data/ResultForm.txt</code>	结果文件格式示例
<code>data/DataFormatExplanation.txt</code>	数据格式说明
<code>Result.txt</code>	程序运行后生成的预测结果文件

注意：`recommender.exe` 必须和 `data` 文件夹放在同一级目录下，否则程序可能找不到 `data/train.txt` 和 `data/test.txt`。

2. 数据格式说明

2.1 train.txt 格式

`train.txt` 中包含已知评分数据，格式如下：

```
<user id>|<number of rating items>  
<item id> <score>  
<item id> <score>  
...
```

例如：

```
1|3
1256 100
3247 60
2644 80
```

含义是：用户 1 对 3 个物品进行了评分。

2.2 test.txt 格式

test.txt 中包含需要预测的用户-物品对，格式如下：

```
<user id>|<number of rating items>
<item id>
<item id>
...
```

例如：

```
1|2
260
1927
```

含义是：需要预测用户 1 对物品 260 和 1927 的评分。

2.3 Result.txt 输出格式

程序会生成 Result.txt，格式与 ResultForm.txt 一致：

```
<user id>|<number of rating items>
<item id> <predicted score>
<item id> <predicted score>
...
```

例如：

```
1|2
260 80
1927 75
```

3. 如何在 PyCharm 中直接运行 Python 代码

3.1 打开项目

用 PyCharm 打开整个项目文件夹，例如：

```
E:\大三下\大数据\推荐系统大作业\任务一
```

不要只打开单个 `.py` 文件，应该打开整个 `任务一` 文件夹。

3.2 确认 Python 环境

代码使用了 Python，并且矩阵分解模型需要 `numpy`。

如果没有安装 `numpy`，在 PyCharm 的 Terminal 中运行：

```
pip install numpy
```

或者：

```
python -m pip install numpy
```

3.3 PyCharm 运行方式一：带参数运行

如果代码中的默认路径仍然是：

```
parser.add_argument("--train", default="train.txt")  
parser.add_argument("--test", default="test.txt")
```

那么需要在 PyCharm 里设置运行参数。

步骤：

```
Run → Edit Configurations → Parameters
```

填写：

```
--train data/train.txt --test data/test.txt --output Result.txt --model baseline
```

然后点击运行。

3.4 PyCharm 运行方式二：直接运行

如果已经把代码中的默认路径改成：

```
parser.add_argument("--train", default="data/train.txt")
parser.add_argument("--test", default="data/test.txt")
parser.add_argument("--output", default="Result.txt")
```

那么可以直接在 PyCharm 中运行 `bigdata_recommender.py`，不需要额外参数。

4. 如何用命令行运行 Python 代码

先进入项目目录：

```
cd /d "E:\大三下\大数据\推荐系统大作业\任务一"
```

然后运行：

```
python bigdata_recommender.py --train data/train.txt --test data/test.txt --output Result.txt --
model baseline
```

也可以使用自动选择模型模式：

```
python bigdata_recommender.py --train data/train.txt --test data/test.txt --output Result.txt --
model auto
```

不过更推荐使用：

```
--model baseline
```

原因是：前期验证中，baseline 模型的 RMSE 表现比矩阵分解模型更好，而且运行更稳定，打包成 `.exe` 后也更容易出现依赖问题。

5. 如何生成 .exe 可执行文件

本项目使用 PyInstaller 将 Python 文件打包成 Windows 可执行文件。

5.1 安装 PyInstaller

在命令行或 PyCharm Terminal 中运行：

```
python -m pip install pyinstaller
```

如果之前因为 `numpy` 报错，建议同时更新依赖：

```
python -m pip install --upgrade pyinstaller pyinstaller-hooks-contrib numpy
```

5.2 进入项目目录

```
cd /d "E:\大三下\大数据\推荐系统大作业\任务一"
```

5.3 删除旧的打包文件

如果之前已经打包过，建议先清理旧文件：

```
rmdir /s /q build  
rmdir /s /q dist  
del recommender.spec
```

如果 `recommender.spec` 不存在，忽略即可。

5.4 打包生成 exe

推荐使用下面这条命令：

```
python -m PyInstaller --onefile --clean --name recommender --collect-all numpy  
bigdata_recommender.py
```

参数含义：

<code>--onefile</code>	打包成单个 exe 文件
<code>--clean</code>	清理临时缓存，避免旧配置影响结果
<code>--name recommender</code>	指定生成的 exe 名字为 <code>recommender.exe</code>
<code>--collect-all numpy</code>	强制收集 <code>numpy</code> 相关依赖，避免运行 exe 时 <code>numpy</code> 报错

打包完成后，生成的 exe 位于：

```
dist/recommender.exe
```

6. 如何运行 exe 文件

6.1 推荐最终运行结构

把 `dist/recommender.exe` 复制出来，放到项目根目录下，最终结构应为：

```
任务一/  
├─ recommender.exe  
└─ data/  
    ├─ train.txt  
    ├─ test.txt  
    ├─ ResultForm.txt  
    └─ DataFormatExplanation.txt
```

6.2 运行 exe

在命令行中进入项目目录：

```
cd /d "E:\大三下\大数据\推荐系统大作业\任务一"
```

然后运行：

```
recommender.exe --train data/train.txt --test data/test.txt --output Result.txt --model baseline
```

运行结束后会生成：

```
Result.txt
```

6.3 如果代码默认路径已改好

如果代码默认读取：

```
data/train.txt  
data/test.txt
```

那么可以直接运行：

```
recommender.exe
```

但是为了避免路径错误，建议仍然用完整参数运行：

```
recommender.exe --train data/train.txt --test data/test.txt --output Result.txt --model baseline
```

7. build 和 dist 文件夹是否需要提交

不需要。

PyInstaller 打包后会生成：

```
build/  
dist/  
recommender.spec
```

其中：

build/	临时编译文件，不需要提交
dist/	存放生成的 exe，不需要整个文件夹
recommender.spec	打包配置文件，一般不需要提交

最终只需要提交或发送：

```
recommender.exe  
bigdata_recommender.py  
data/  
Result.txt
```

如果给老师或同学检查，建议也附上本说明文档。

8. 代码整体逻辑说明

代码主要分为以下几个部分：

1. 读取 train.txt
2. 读取 test.txt
3. 统计数据集基本信息
4. 训练推荐模型
5. 预测 test.txt 中的评分
6. 按 ResultForm.txt 的格式输出 Result.txt

8.1 数据读取

代码中的 `read_train()` 负责读取训练集。

它会吧 `train.txt` 转换成如下三元组：

```
(user_id, item_id, score)
```

例如：

```
1 1256 100
```

会被存成：

```
(1, 1256, 100.0)
```

代码中的 `read_test_groups()` 负责读取测试集。

它会保留原始的用户分组结构，例如：

```
(user_id, [item1, item2, item3])
```

这样输出 `Result.txt` 时可以保持和 `test.txt` 一样的顺序和格式。

8.2 结果输出

`write_result()` 负责生成最终结果文件。

核心逻辑是：

```
for user, items in test_groups:
    写入 user|物品数量
    for item in items:
        预测 user 对 item 的评分
        写入 item predicted_score
```

预测分数会被限制在评分范围内：

```
10 到 100
```

并默认四舍五入成整数。

8.3 数据集统计

代码中的 `basic_stats()` 会统计：

训练集评分数量
训练集用户数量
训练集物品数量
测试集预测对数量
测试集用户数量
测试集物品数量
冷启动用户数量
冷启动物品数量
评分最小值
评分最大值
评分均值
评分矩阵稀疏度

这些信息可以直接用于报告中的“Basic statistics of the dataset”部分。

其中稀疏度计算公式为：

$$\text{sparsity} = 1 - \text{已知评分数量} / (\text{用户数量} \times \text{物品数量})$$

9. 使用的推荐模型

代码中实现了三类模型：

1. Mean Model
2. Baseline Bias Model
3. Matrix Factorization Model

另外还有一个 `auto` 模式，会在验证集上比较模型 RMSE，然后选择表现最好的模型。

10. Model 1: Mean Model

Mean Model 是最基础的平均分模型，用作简单基准模型。

它包括：

全局平均分 `global_mean`
用户平均分 `user_mean`
物品平均分 `item_mean`

基本思想是：

如果用户和物品都出现过，则综合使用全局均值、用户均值和物品均值：

$$\text{pred} = 0.50 \times \text{global_mean} + 0.25 \times \text{user_mean} + 0.25 \times \text{item_mean}$$

如果只有物品出现过：

$$\text{pred} = 0.35 \times \text{global_mean} + 0.65 \times \text{item_mean}$$

如果只有用户出现过：

$$\text{pred} = 0.35 \times \text{global_mean} + 0.65 \times \text{user_mean}$$

如果用户和物品都没有出现过：

$$\text{pred} = \text{global_mean}$$

这个模型的优点是：

实现简单
速度快
可以处理冷启动用户或冷启动物品

缺点是：

不能建模用户和物品之间的复杂交互关系
预测能力有限

11. Model 2: Baseline Bias Model

Baseline Bias Model 是本项目最终推荐使用的模型。

它的预测公式是：

$$r_{\hat{u}i} = \mu + b_u + b_i$$

其中：

$r_{\hat{u}i}$ 表示用户 u 对物品 i 的预测评分
 μ 表示全局平均评分
 b_u 表示用户偏置
 b_i 表示物品偏置

含义是：

一个评分 = 全局平均水平 + 用户个人打分倾向 + 物品自身受欢迎程度

例如：

有些用户整体打分偏高，所以 b_u 为正
有些用户整体打分偏低，所以 b_u 为负
有些物品普遍评分较高，所以 b_i 为正
有些物品普遍评分较低，所以 b_i 为负

该模型使用 SGD，即随机梯度下降进行训练。

每次训练时，先计算预测误差：

$$err = r_{ui} - r_{\hat{u}i}$$

然后更新：

$$\begin{aligned} b_u &= b_u + lr \times (err - reg \times b_u) \\ b_i &= b_i + lr \times (err - reg \times b_i) \end{aligned}$$

其中：

lr 是学习率
 reg 是正则化系数，用于防止过拟合

优点：

比简单平均模型更准确
训练速度快
内存消耗低

可解释性强
适合写进报告

缺点：

只能建模用户和物品的整体偏置
无法捕捉更复杂的兴趣特征交互

12. Model 3: Matrix Factorization Model

Matrix Factorization，即矩阵分解模型，是推荐系统中的经典协同过滤方法。

代码中使用的是带 bias 的矩阵分解模型：

$$r_{\hat{u}i} = \mu + b_u + b_i + p_u^T q_i$$

其中：

μ	全局平均评分
b_u	用户偏置
b_i	物品偏置
p_u	用户 u 的隐向量
q_i	物品 i 的隐向量
$p_u^T q_i$	用户与物品在隐空间中的匹配程度

直观理解：

p_u 表示用户的潜在兴趣特征
 q_i 表示物品的潜在属性特征
 $p_u^T q_i$ 越大，说明用户越可能喜欢该物品

例如在电影推荐中，隐向量可能隐含表示：

动作片偏好
喜剧片偏好
科幻片偏好
经典电影偏好

虽然这些特征不是人工指定的，但模型可以从评分数据中自动学习。

矩阵分解模型同样使用 SGD 训练。

优点：

能够学习用户和物品之间的潜在交互关系
理论上表达能力强
是推荐系统中非常经典的方法

缺点：

训练时间更长
依赖 numpy
参数更敏感
在本数据集默认参数下，效果不一定优于 baseline
打包成 exe 时可能出现 numpy 依赖问题

13. Auto 模式

如果运行：

```
--model auto
```

程序会：

1. 从 train.txt 中划分一部分作为 validation set
2. 分别训练 mean、baseline、mf 三种模型
3. 计算每个模型在验证集上的 RMSE
4. 选择 RMSE 最低的模型
5. 在完整训练集上重新训练该模型
6. 对 test.txt 生成预测结果

RMSE 公式为：

$$\text{RMSE} = \sqrt{\sum (r_{\text{hat}}_{ui} - r_{ui})^2 / |R|}$$

RMSE 越小，说明预测结果越接近真实评分。

前期测试结果中，baseline 模型表现最好，因此最终建议运行：

```
--model baseline
```

而不是盲目使用矩阵分解模型。

14. 为什么最终推荐 baseline 模型

虽然矩阵分解模型理论上更复杂，但复杂不等于一定更好。

本项目中推荐使用 baseline 模型，原因是：

1. 验证集 RMSE 更低
2. 训练速度更快
3. 内存消耗更小
4. 结果更稳定
5. 模型可解释性更强
6. 更适合写入课程报告
7. 打包成 exe 后出错概率更低

这也是报告里可以重点分析的地方：

在当前数据集和默认参数设置下，简单而稳定的 bias baseline 模型比矩阵分解模型取得了更好的验证集 RMSE。这说明复杂模型并不一定在所有数据集上都优于简单模型，模型效果还受到数据规模、稀疏性、参数设置和训练轮数等因素影响。

15. 报告中可以这样描述模型

可以写：

This project implements three rating prediction models: a mean-based baseline model, a bias baseline model, and a biased matrix factorization model. The mean-based model uses global, user, and item average ratings as fallback predictions. The bias baseline model decomposes each rating into the global mean, user bias, and item bias. The matrix factorization model further extends the bias baseline by introducing latent user and item vectors to capture hidden user-item interactions.

中文说明：

本项目实现了三类评分预测模型：基于平均值的基准模型、Bias Baseline 模型，以及带偏置项的矩阵分解模型。平均值模型主要使用全局均值、用户均值和物品均值作为预测依据；Bias Baseline 模型将评分分解为全局平均分、用户偏置和物品偏置；矩阵分解模型则进一步加入用户和物品的隐向量，用于捕捉潜在的用户-物品交互关系。

16. 常见问题

16.1 exe 双击闪退

这通常不是程序错误，而是窗口运行结束后自动关闭。

建议在命令行中运行：

```
recommender.exe --train data/train.txt --test data/test.txt --output Result.txt --model baseline
```

这样可以看到完整输出。

16.2 找不到 train.txt 或 test.txt

检查目录结构是否正确。

正确结构必须是：

```
recommender.exe  
data/train.txt  
data/test.txt
```

也就是说，`data` 文件夹必须和 `recommender.exe` 在同一级目录。

16.3 numpy 报错

如果 exe 运行时报类似错误：

```
Importing the numpy C-extensions failed  
No module named numpy._core._exceptions
```

说明 PyInstaller 打包 numpy 依赖失败。

解决方法是重新打包：

```
python -m PyInstaller --onefile --clean --name recommender --collect-all numpy  
bigdata_recommender.py
```

如果仍然失败，可以尝试降低 numpy 版本：